FastBite

(notre application de livraison de repas, type Deliveroo)

TP6 — Interface graphique Swing et programmation événementielle

[Prénom NOM 1] · [Prénom NOM 2] · [Prénom NOM 3]

Programmation orientée objet en Java — ECE Paris, Ing2

Enseignant : J.-P. Segado · octobre 2026

Sommaire

1 **Q1 — Maquette (wireframes)**
les 3 fenêtres dessinées avant de coder (1 slide)

2 **Q2 — Storyboard**
la navigation entre les pages, les erreurs, les fichiers (1 slide)

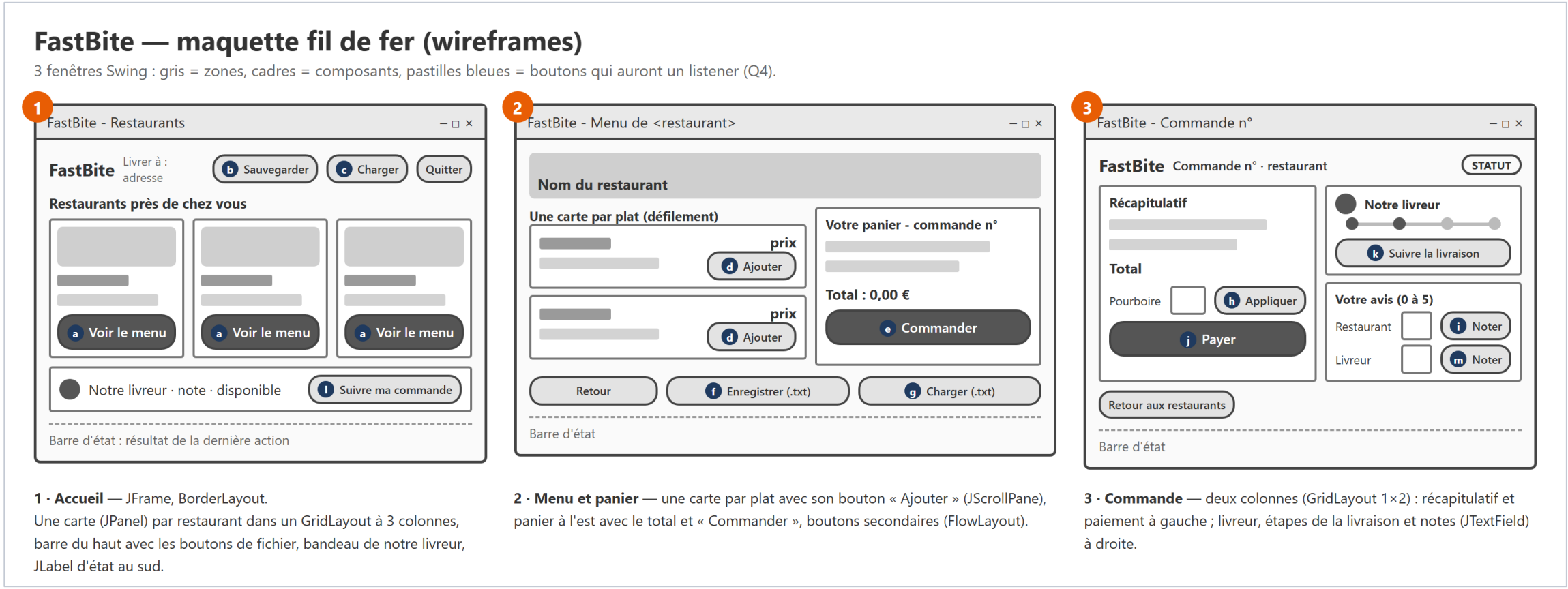
3 **Q3 — Fenêtres Swing**
code commenté et fenêtres obtenues (2 slides)

4 **Q4 — Listeners, exceptions, fichiers**
clics, erreurs affichées, .txt et .ser (3 slides)

5 **Bilan, sources et IA**
ce qui marche, limites, outils utilisés

Point de départ : le code du TP5 (classes métier, 9 exceptions, fichier texte, sérialisation). Le TP6 n'ajoute que des classes d'interface ; le métier n'est pas réécrit.

Q1 — Maquette fil de fer (wireframes)



1 · Accueil — JFrame, BorderLayout.
Une carte (JPanel) par restaurant dans un GridLayout à 3 colonnes, barre du haut avec les boutons de fichier, bandeau de notre livreur, JLabel d'état au sud.

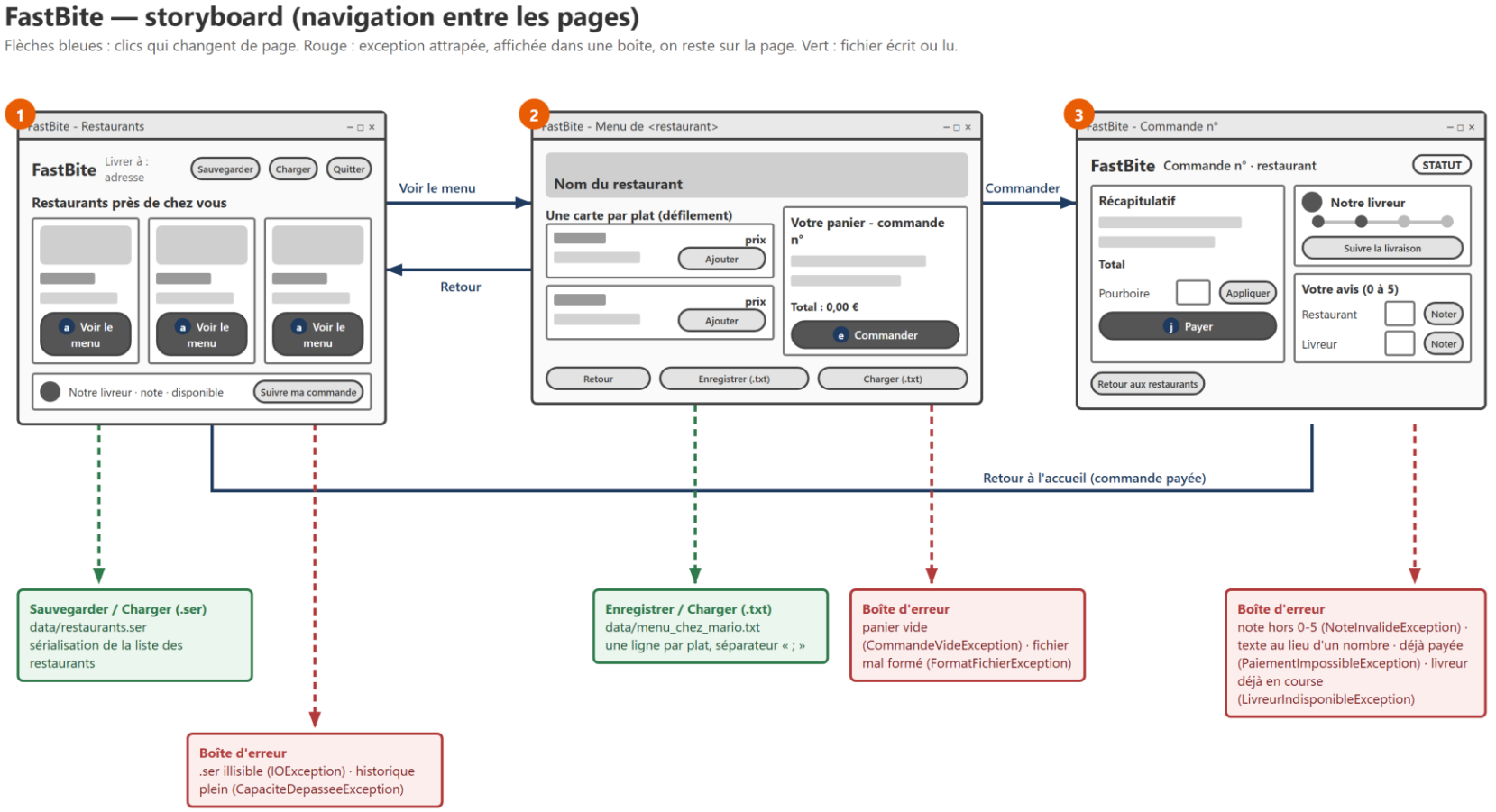
2 · Menu et panier — une carte par plat avec son bouton « Ajouter » (JScrollPane), panier à l'est avec le total et « Commander », boutons secondaires (FlowLayout).

3 · Commande — deux colonnes (GridLayout 1×2) : récapitulatif et paiement à gauche ; livreur, étapes de la livraison et notes (JTextField) à droite.

Capture 1 — maquette des 3 fenêtres, générée par IA à partir de nos TP2 à TP5

- Pourquoi 3 fenêtres ? Elles suivent le cas d'utilisation principal du TP2 : choisir un restaurant, composer sa commande, payer et noter.
- Présentation inspirée des applications de livraison (cartes, panier à droite, suivi du livreur) ; chaque zone est un composant Swing (JPanel, JLabel, JTextField, JButton), les pastilles bleues sont les boutons qui auront un listener.

Q2 — Storyboard : navigation entre les pages



Capture 2 — storyboard généré par IA

Lecture

- Bleu : un clic change de page (Voir le menu, Commander, Retour, Suivre ma commande).
- Rouge : une exception est attrapée ; une boîte d'erreur s'ouvre et on reste sur la page.
- Vert : un fichier est écrit ou lu (.ser sur l'accueil, .txt sur le menu).

Choix

- Une seule fenêtre visible à la fois : l'accueil est caché, pas détruit, pour y revenir.
- Les fenêtres partagent un objet Donnees (restaurants, client, livreur, commande en cours de livraison).

Q3 — Construire une fenêtre Swing : le code

• FenetreAccueil.java

```java
public class FenetreAccueil extends JFrame implements ActionListener {
```

• FenetreAccueil.java

```java
        // --- CENTRE : titre puis grille de cartes (GridLayout = cases de meme taille)
        JPanel titres = Style.groupe(new GridLayout(2, 1, 0, 2));
        titres.add(Style.texte("Restaurants près de chez vous", Font.BOLD, 24, Style.NOIR));
        titres.add(compteur);
        JPanel enveloppe = Style.groupe(new BorderLayout());
        enveloppe.add(grille, BorderLayout.NORTH);            // au nord : les cartes gardent leur hauteur
        JScrollPane defilement = new JScrollPane(enveloppe);  // barre de defilement s'il y a beaucoup de restaurants
        defilement.setBorder(BorderFactory.createEmptyBorder());
        defilement.setViewportBorder(null);
        defilement.setOpaque(false);
        defilement.getViewport().setOpaque(false);
        defilement.getVerticalScrollBar().setUnitIncrement(16);
```

• FenetreAccueil.java

```java
        // BorderLayout : 5 zones (NORTH, CENTER, EAST, SOUTH, WEST)
        JPanel centre = new JPanel(new BorderLayout(0, 14));
        centre.setBackground(Style.FOND);
        centre.setBorder(BorderFactory.createEmptyBorder(18, 22, 10, 22));
        centre.add(titres, BorderLayout.NORTH);
        centre.add(defilement, BorderLayout.CENTER);
        centre.add(sud, BorderLayout.SOUTH);
        JPanel contenu = new JPanel(new BorderLayout());
        contenu.add(nord, BorderLayout.NORTH);
        contenu.add(centre, BorderLayout.CENTER);
        setContentPane(contenu);

        // --- LISTENERS : la fenetre s'abonne aux clics de ses boutons
        boutonSauver.addActionListener(this);
        boutonCharger.addActionListener(this);
        boutonQuitter.addActionListener(this);
        boutonSuivi.addActionListener(this);

        rafraichir();
        setSize(1000, 660);
        setLocationRelativeTo(null);              // centre de l'ecran
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Capture 3 — FenetreAccueil : composants, layouts, abonnement aux boutons

• FastBiteGUI.java

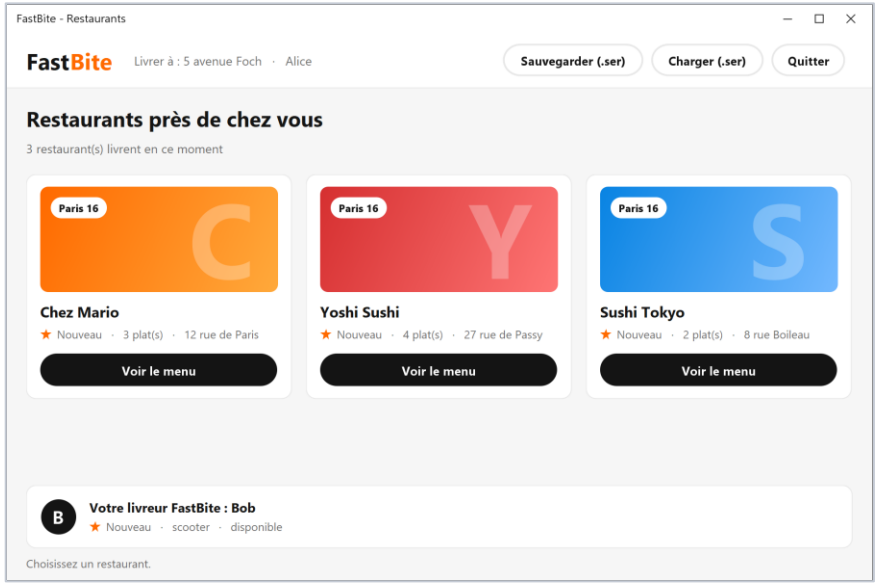
```java
    public static void main(String[] args) {
        installerApparence();
        // Swing demande que les fenetres soient creees dans son propre fil (thread)
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                Donnees donnees = new Donnees();
                FenetreAccueil accueil = new FenetreAccueil(donnees);
                try {
                    donnees.creerJeuDEssai();
                    accueil.rafraichir();
                } catch (CapaciteDepasseeException e) {
                    Messages.erreur(accueil, "Demarrage", e.getMessage());
                }
                accueil.setVisible(true);
            }
        });
    }
```

Capture 4 — FastBiteGUI.main : lancement

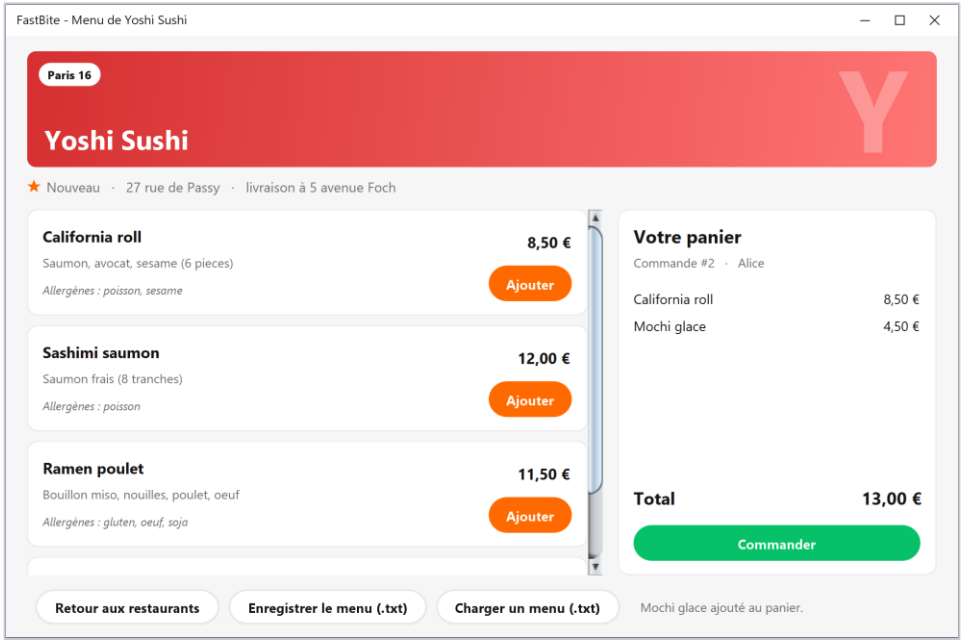
Ce que fait ce code

- La classe hérite de JFrame : c'est une fenêtre.
- BorderLayout : 5 zones. GridLayout à 3 colonnes : une carte par restaurant, toutes de même taille.
- JScrollPane autour de la grille : barre de défilement s'il y a beaucoup de restaurants.
- addActionListener(this) : la fenêtre écoute ses boutons. Notre classe Style redéfinit paintComponent (boutons arrondis, cartes).
- invokeLater : Swing crée les fenêtres dans son propre fil d'exécution.

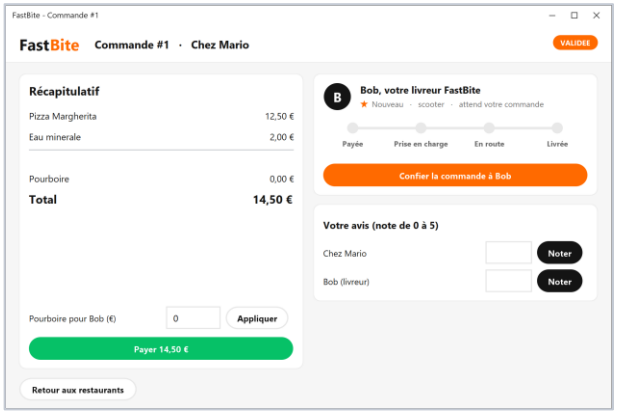
Q3 — Les 3 fenêtres obtenues



Capture 5 — page 1 : accueil (une carte par restaurant, notre livreur)



Capture 6 — page 2 : menu (une carte par plat) et panier



Capture 7 — page 3 : commande

- Fidèle à la maquette : mêmes zones, mêmes boutons.
- Les cartes sont construites à partir des vrais objets Restaurant et Plat (TP3) : Yoshi Sushi, ajouté dans les données, apparaît sans toucher aux fenêtres.
- Charte FastBite (orange, noir, vert) regroupée dans la classe Style ; présentation inspirée des applications de livraison.

Q4 — Listeners : un clic met à jour l'affichage

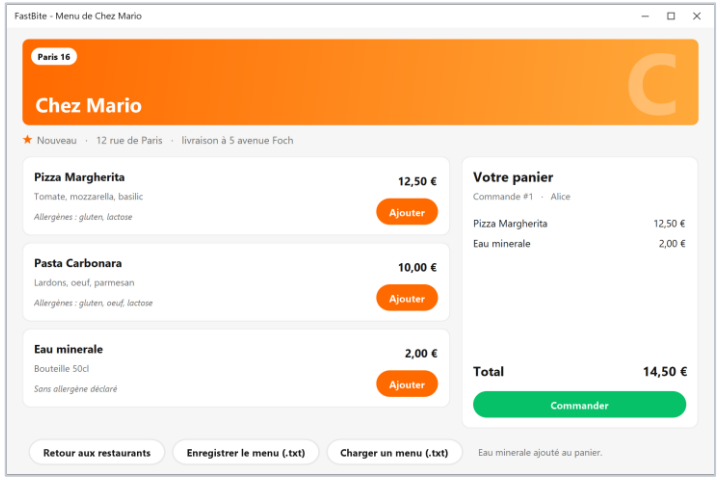
• FenetreAccueil.java

```java
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();
    int indice = boutonsMenu.indexOf(source);     // -1 si ce n'est pas un bouton « Voir le menu »
    if (indice >= 0) {
        ouvrirMenu(indice);
    } else if (source == boutonSauver) {
        sauvegarder();
    } else if (source == boutonCharger) {
        charger();
    } else if (source == boutonSuivi) {
        ouvrirSuivi();
    } else if (source == boutonQuitter) {
        dispose();
    }
}
```

• FenetreMenu.java

```java
private void ajouterAuPanier(int numero) {
    Plat plat = restaurant.getMenu()[numero];
    try {
        commande.ajouterPlat(plat);
        rafraichirPanier();                                  // mise a jour de l'affichage
        etat.setText(plat.getNom() + " ajouté au panier.");
    } catch (PlatIndisponibleException ex) {
        Messages.erreur(this, "Plat indisponible", ex.getMessage());
    }
}
```

Capture 8 — actionPerformed + getSource, puis le traitement d'un clic



Capture 9 — test : 2 clics « Ajouter » → total 14,50 €

• FenetreCommande.java

```java
// --- LISTENERS (classes anonymes)
boutonPourboire.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) { appliquerPourboire(); }
});
boutonNoter.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) { noter(); }
```

Capture 10 — autre écriture : classe anonyme (page 3)

- Deux façons vues au cours 6 : la fenêtre implémente ActionListener (getSource dit quel bouton ; sa position dans la liste des boutons donne le plat), ou une classe anonyme par bouton.
- Le listener modifie le modèle (commande.ajouterPlat) puis rafraîchit la vue.

Q4 — Les exceptions du TP5 dans l'interface

• FenetreMenu.java

```java
private void validerCommande() {
    try {
        commande.valider();
        restaurant.recevoirCommande(commande);
        FenetreCommande page = new FenetreCommande(accueil, donnees, commande);
        page.setVisible(true);
        dispose();                                          // on ferme la page 2
    } catch (CommandeVideException ex) {
        Messages.erreur(this, "Commande vide", ex.getMessage());
    }
}
```

• FenetreCommande.java

```java
private void noter() {
    Restaurant r = commande.getRestaurant();
    try {
        double note = Double.parseDouble(champNote.getText().trim().replace(',', '.'));
        r.ajouterNote(note);
        etat.setText("Merci ! Note " + note + " enregistrée pour " + r.getNom()
                + " (moyenne : " + Style.note(r) + " / 5).");
    } catch (NumberFormatException ex) {
        Messages.erreur(this, "Note invalide", "\"" + champNote.getText() + "\" n'est pas un nombre.");
    } catch (NoteInvalideException ex) {
        Messages.erreur(this, "Note invalide", ex.getMessage());
    }
}
```

Capture 11 — try / catch dans les listeners : le message de l'exception va dans une boîte



Capture 12 — Valider avec un panier vide : CommandeVideException



Capture 13 — note 7 : NoteInvalideException



Capture 14 — 2e clic sur Payer : PaiementImpossibleException



Capture 14 bis — 2e commande pendant une livraison : LivreurIndisponibleException

- Les classes métier lèvent ou propagent (TP5) ; c'est le listener qui attrape et affiche.
- Après une erreur, la fenêtre reste utilisable et les données sont intactes.
- Nouveau cas : NumberFormatException si le champ ne contient pas un nombre.

Q4 — Fichier texte et sérialisation depuis l'interface

FenetreMenu.java

```java
public void chargerMenu(String fichier) {
    Restaurant relu = new Restaurant(restaurant.getNom(), restaurant.getAdresse(), restaurant.getZoneLivraison());
    try {
        relu.chargerMenu(fichier);
        donnees.getRestaurants().set(indice, relu);          // remplace l'ancien restaurant
        restaurant = relu;
        commande = new Commande(commande.getNumero(), donnees.getClient(), relu);   // panier vide
        remplirMenu();
        rafraichirPanier();
        etat.setText(relu.getNbPlats() + " plat(s) lu(s) depuis " + fichier);
    } catch (FormatFichierException ex) {
        Messages.erreur(this, "Fichier incorrect", ex.getMessage());
    } catch (IOException ex) {
        Messages.erreur(this, "Lecture impossible", ex.getMessage());
    } catch (CapaciteDepasseeException ex) {
        Messages.erreur(this, "Menu trop long", ex.getMessage());
    }
}
```

FenetreAccueil.java

```java
private void charger() {
    try {
        ArrayList<Restaurant> lus = Restaurant.chargerRestaurants(Donnees.FICHIER_SER);
        donnees.setRestaurants(lus);
        rafraichir();
        etat.setText(lus.size() + " restaurant(s) relu(s) depuis " + Donnees.FICHIER_SER);
    } catch (IOException ex) {
        // fichier absent, illisible ou qui n'est pas un flux d'objets Java
        Messages.erreur(this, "Chargement impossible", "Lecture de " + Donnees.FICHIER_SER + " : " + ex.getMessage());
    } catch (ClassNotFoundException ex) {
        Messages.erreur(this, "Chargement impossible", "Classe inconnue dans le fichier : " + ex.getMessage());
    }
}
```

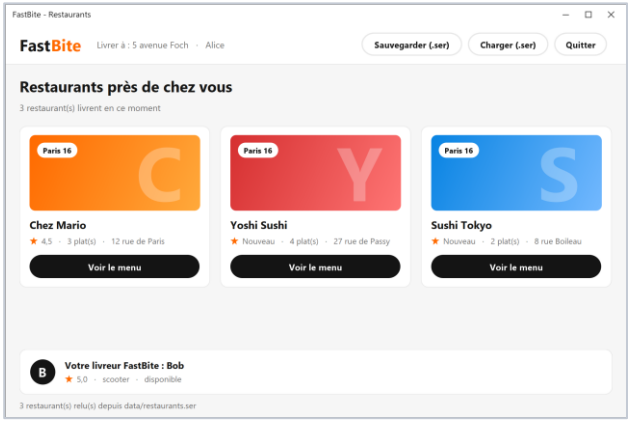
Capture 15 — chargerMenu (.txt) et charger (.ser) : appel des méthodes du TP5 + catch

data/menu_chez_mario.txt - Bloc-notes

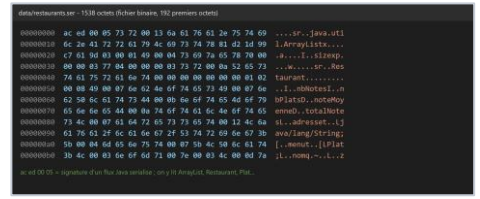
```
Pizza Margherita;12.5;Tomate, mozzarella, basilic;gluten,lactose
Pasta Carbonara;10.0;Lardons, oeuf, parmesan;gluten,oeuf,lactose
Eau minerale;2.0;Bouteille 50cl;
```

UTF-8 | 3 lignes | separateur : point-virgule

Capture 16 — data/menu_chez_mario.txt écrit par le bouton « Enregistrer »



Capture 17 — après « Charger (.ser) »



Capture 18 — le .ser (binaire)



Capture 19 — fichier .txt mal formé : FormatFichierException



Capture 20 — fichier .ser corrompu : IOException

- Le menu est lu dans un restaurant neuf : si le fichier est faux, l'écran n'est pas modifié.
- Test : accueil vidé puis « Charger » → 3 restaurants, menus et note 4,5 retrouvés.

Bilan, sources et IA

Bilan

• Fait : maquette, storyboard, 3 fenêtres Swing reliées, un bouton avec listener sur chaque carte, exceptions affichées, fichiers .txt et .ser, suivi de la livraison par notre livreur.

• Vérifié : TestsTP6 simule les clics (doClick) : 39 vérifications passent ; les tests du TP5 passent toujours.

• Appris : JFrame et layouts, ActionListener (this ou classe anonyme), séparer l'interface du métier, le fil d'exécution de Swing.

• Bug trouvé grâce aux tests : après la lecture d'un .ser corrompu, le fichier restait ouvert (TP5). Corrigé avec deux ressources dans le try.

• Limites : un seul client et un seul livreur, pas d'écran de connexion ni de photos des plats ; les images de fenêtres sont des rendus du programme de test.

• Améliorations : séparer en Modèle-Vue-Contrôleur (TP7), plusieurs livreurs, retirer un plat du panier.

Sources

• J.-P. Segado, TP6 et cours 6 « Interface graphique Swing, AWT, Listeners » (ECE).

• Documentation Java SE (Oracle) : JFrame, ActionListener, ActionEvent, JOptionPane.

• Notre code du TP5 (exceptions, fichier, sérialisation).

IA utilisée

• Claude (Anthropic) : maquette, storyboard, code Swing, tests et slides générés par IA. Part d'aide de l'IA : à indiquer par l'équipe.

Prompt utilisé :

• « fais le tp6 dans la suite du tp5 fastbite », puis « inspire toi uber eats [...] rajoute le restaurant yoshi sushi » (sujet du TP6 et code du TP5 fournis à l'IA)